

КУРСОВАЯ РАБОТА

по дисциплине: «Создание программного обеспечения»

на тему:

«Разработка информационной системы управления лифтами»

Выполнил:

студент

Группа:

Проверил:

Город:

Год:

2026

Москва, 2026

Аннотация

В курсовой работе разработана информационная система управления лифтами, предназначенная для приема пользовательских вызовов, автоматического назначения оптимальной кабины, мониторинга состояния лифтов, ведения журнала событий и настройки интерфейса. Система реализована как распределенный монолит: клиентская часть выполнена на Vue 3, серверная часть представляет собой REST API на Node.js/Express, база данных - PostgreSQL. В проекте применены CRUD-операции, ролевая модель доступа, JWT-авторизация, UML-диаграммы, ERD в нотации Crow's Foot, а также паттерны Strategy, Observer и Repository.

Практическая часть содержит исходный код frontend и backend, документацию API, схему базы данных, мокапы интерфейса и материалы для защиты. Проект ориентирован на требования эргономики взаимодействия человек-система: единообразие интерфейса, понятная обратная связь, настройка интерфейса, доступность и простота освоения.

Задание на курсовую работу

Необходимо разработать информационную систему «Система управления лифтами» с использованием методологии Waterfall. Система должна иметь отдельный frontend на веб-фреймворке и отдельный backend, предоставляющий REST API в формате JSON. В системе должны быть реализованы CRUD-операции, UML-диаграммы, ERD в нотации Crow's Foot, минимум три паттерна проектирования, описание тестирования и технической документации.

Ключевое требование	Реализация в проекте
Единообразие интерфейса	Единая навигационная панель, одинаковые карточки статусов, общие

	компоненты Vuetify.
Своевременная обратная связь	Сообщения об успешных операциях, ошибки валидации, статусы вызовов и лифтов.
Настройка интерфейса	Настройки темы, размера текста, режима доступности и звуковой обратной связи.
Доступность	Контрастная тема, крупный текст, простые формулировки, крупные кнопки.
Интуитивность	Минимум экранов, понятные действия: «Вызвать», «Изменить», «Удалить», «Сохранить».

Содержание

Введение

1 Анализ предметной области

2 Методология Waterfall

3 Анализ аналогов и конкурентов

4 Требования к системе

5 Технические и технологические нововведения

6 Архитектура распределенного монолита

7 Проектирование базы данных

8 UML-проектирование

9 Паттерны проектирования

10 Проектирование интерфейса

11 REST API и CRUD

12 Реализация приложения

13 Тестирование

14 Введение в эксплуатацию

Заключение

Список источников

Приложения

Введение

Современные здания часто оснащаются несколькими лифтами, которые должны работать согласованно, безопасно и удобно для пользователей. При большом количестве пассажиров возникает необходимость в автоматизированной системе, которая принимает вызовы, анализирует положение лифтов, назначает подходящую кабину и предоставляет пользователю понятную информацию о результате действия.

Актуальность темы связана с тем, что качество работы лифтовой системы влияет на скорость перемещения людей внутри здания, безопасность эксплуатации, удобство диспетчерского контроля и эффективность технического обслуживания. При отсутствии централизованной системы управления пользователи не получают своевременную обратную связь, диспетчеры не видят полной картины работы оборудования, а события неисправностей сложно анализировать.

Цель курсовой работы - разработать информационную систему управления лифтами, обеспечивающую прием вызовов, автоматическое назначение лифта, мониторинг состояния оборудования, ведение журнала событий и настройку интерфейса под потребности пользователя.

Для достижения цели необходимо решить следующие задачи: проанализировать предметную область и бизнес-процессы; определить роли пользователей и объекты данных; спроектировать архитектуру распределенного монолита; разработать UML-диаграммы и ERD; реализовать frontend и backend; применить паттерны проектирования; провести тестирование основных функций.

Объект исследования - процесс управления лифтами в здании. Предмет исследования - программные средства приема вызовов, диспетчеризации, мониторинга и настройки интерфейса пользователя.

1 Анализ предметной области

Предметная область включает процессы взаимодействия пассажиров, диспетчеров, администраторов и технических специалистов с системой управления лифтами. В типовом сценарии пассажир выбирает этаж отправления и этаж назначения, система создает вызов, анализирует доступные лифты и назначает кабину с учетом выбранного алгоритма диспетчеризации.

Диспетчер контролирует состояние лифтов, видит ошибки, активные вызовы и журнал событий. Администратор управляет пользователями, ролями, лифтами, этажами и правилами диспетчеризации. Технический специалист получает информацию о неисправностях и может фиксировать события обслуживания.

Роль	Основные действия	Права доступа
Пассажир	Вызов лифта, выбор этажа, просмотр статуса, настройка интерфейса.	Доступ к вызовам и личным настройкам.
Диспетчер	Мониторинг лифтов, просмотр журнала, изменение статусов при необходимости.	Доступ к панели мониторинга и событиям.
Администратор	CRUD пользователей, лифтов, этажей, правил диспетчеризации.	Полный доступ к управлению системой.
Техник	Просмотр состояния, фиксация неисправностей и событий обслуживания.	Доступ к журналу и техническим статусам.

1.1 Анализируемые задачи

№	Задача анализа	Что подверглось анализу	Результат
1	Пользовательские сценарии	Действия пассажира, диспетчера, администратора и техника.	Определены роли и границы доступа.
2	Бизнес-процессы	Прием вызова, назначение лифта, обновление статуса, журналирование.	Выделены основные процессы приложения.
3	Объекты данных	Пользователи, роли, лифты, этажи, вызовы, события, настройки.	Сформирована логическая модель БД.
4	Интерфейсные требования	Единообразие, обратная связь, доступность, настройка интерфейса.	Определены UI-компоненты и режим доступности.
5	Техническая реализация	Frontend, backend, REST API, база	Выбран стек Vue 3 + Express +

		данных, авторизация.	PostgreSQL.
--	--	-------------------------	-------------

2 Методология Waterfall

Разработка выполняется по каскадной методологии Waterfall, при которой этапы проекта проходят последовательно. Такой подход подходит для курсовой работы, поскольку требования заранее сформулированы, а результат должен включать анализ, проектирование, разработку, тестирование и описание ввода в эксплуатацию.

Этап Waterfall	Содержание работ	Результат
Анализ требований	Изучение предметной области, ролей, CRUD-функций, требований к интерфейсу.	Список функциональных и нефункциональных требований.
Проектирование	Разработка архитектуры, UML, ERD, REST API, макетов интерфейса.	Диаграммы и проектные решения.
Реализация	Создание frontend на Vue и backend REST API, настройка БД.	Работающий прототип системы.
Тестирование	Проверка CRUD, авторизации, вызовов, ошибок и интерфейса.	Тест-кейсы и результаты проверки.
Ввод в эксплуатацию	Подготовка README, Docker Compose,	Документация для

	инструкции запуска.	развертывания.
--	---------------------	----------------

3 Анализ аналогов и конкурентов

На рынке существуют решения диспетчеризации лифтов, которые применяются в современных зданиях и бизнес-центрах. Обычно такие системы используют принцип назначения кабины до посадки пассажира, группировку людей по направлению движения, мониторинг оборудования и интеграцию с системами безопасности здания.

Аналог	Характерные возможности	Что используется в проекте
KONE Destination	Назначение кабины на основании этажа назначения.	Идея предварительного выбора этажа назначения.
Otis Compass	Группировка пассажиров и оптимизация поездок.	Группировка по направлению движения.
Schindler PORT	Диспетчеризация, навигация, доступность и интеграция.	Понятная обратная связь и контроль доступа.
TK Elevator AGILE	Сокращение остановок и оптимизация потоков.	Алгоритм минимизации времени ожидания.

Отличие разрабатываемой системы заключается в учебной направленности и открытой архитектуре: проект содержит понятную REST API-

структуру, CRUD-модули, логирование, пользовательские настройки и демонстрационные алгоритмы диспетчеризации.

4 Требования к системе

4.1 Функциональные требования

Код	Требование	Приоритет
FR-01	Пользователь должен иметь возможность авторизоваться в системе.	Высокий
FR-02	Пассажир должен иметь возможность создать вызов лифта.	Высокий
FR-03	Система должна назначать оптимальный лифт для вызова.	Высокий
FR-04	Администратор должен выполнять CRUD для пользователей, этажей и лифтов.	Высокий
FR-05	Диспетчер должен просматривать текущее состояние лифтов.	Высокий
FR-06	Система должна вести журнал событий и	Средний

	ошибок.	
FR-07	Пользователь должен настраивать тему, размер текста и режим доступности.	Средний
FR-08	Система должна предоставлять REST API в формате JSON.	Высокий

4.2 Нефункциональные требования

Категория	Требование	Реализация
Удобство	Интерфейс должен быть единообразным и простым.	Единая сетка, навигация и компоненты.
Обратная связь	Пользователь должен видеть результат действия.	Alert-сообщения, статусы, цветовые индикаторы.
Доступность	Система должна быть удобна для пользователей с разными возможностями.	Крупный шрифт, контраст, понятные подписи.
Безопасность	Доступ должен разграничиваться по ролям.	JWT и middleware authorize.

Надежность	Ошибки должны обрабатываться без сбоя приложения.	Глобальный errorHandler и сообщения в UI.
Производительность	Ответ API должен быть быстрым для типовых операций.	Простые SQL-запросы через Prisma.

5 Технические и технологические нововведения

Нововведение	Описание	Какую задачу решает
Алгоритм выбора оптимального лифта	Сервис DispatchService выбирает кабину по стратегии.	Сокращение времени ожидания и формализация диспетчеризации.
Destination dispatch	Пользователь сразу выбирает этаж назначения.	Система получает больше данных для оптимального назначения.
Режим доступности	Крупный текст, контрастная тема, понятные сообщения.	Равные возможности для пользователей с разными потребностями.
Журнал событий	Каждое важное действие записывается в таблицу events.	Аудит, анализ ошибок и контроль безопасности.
REST API	Frontend обращается к backend через JSON-	Разделение интерфейса

	запросы.	и бизнес-логики.
JWT-авторизация	Проверка токена и роли пользователя.	Защита данных и разграничение прав.

6 Архитектура распределенного монолита

Проект реализован как распределенный монолит. Это означает, что frontend и backend запускаются как отдельные приложения и взаимодействуют через REST API, но логически составляют одну информационную систему с единой базой данных и общей предметной моделью.



Рисунок 1 - Архитектура распределенного монолита

Frontend на Vue отвечает за отображение интерфейса, маршрутизацию, хранение состояния авторизации и отправку HTTP-запросов. Backend на Express принимает запросы, проверяет права доступа, выполняет бизнес-логику и работает с PostgreSQL через Prisma ORM.

Компонент	Назначение	Технологии
Frontend	Интерфейс пользователя и диспетчера.	Vue 3, Vuetify, Pinia, Vue Router, Axios.
Backend	REST API, авторизация, бизнес-логика.	Node.js, Express, Prisma, JWT.
База данных	Хранение структурированных данных.	PostgreSQL.

Docker Compose	Запуск всех частей проекта.	Docker, контейнеры frontend/backend/db.
----------------	-----------------------------	---

7 Проектирование базы данных

База данных хранит роли, пользователей, настройки интерфейса, этажи, лифты, вызовы, события и правила диспетчеризации. Такая структура позволяет реализовать CRUD-функции и обеспечить связь вызовов с конкретными пользователями, этажами и лифтами.

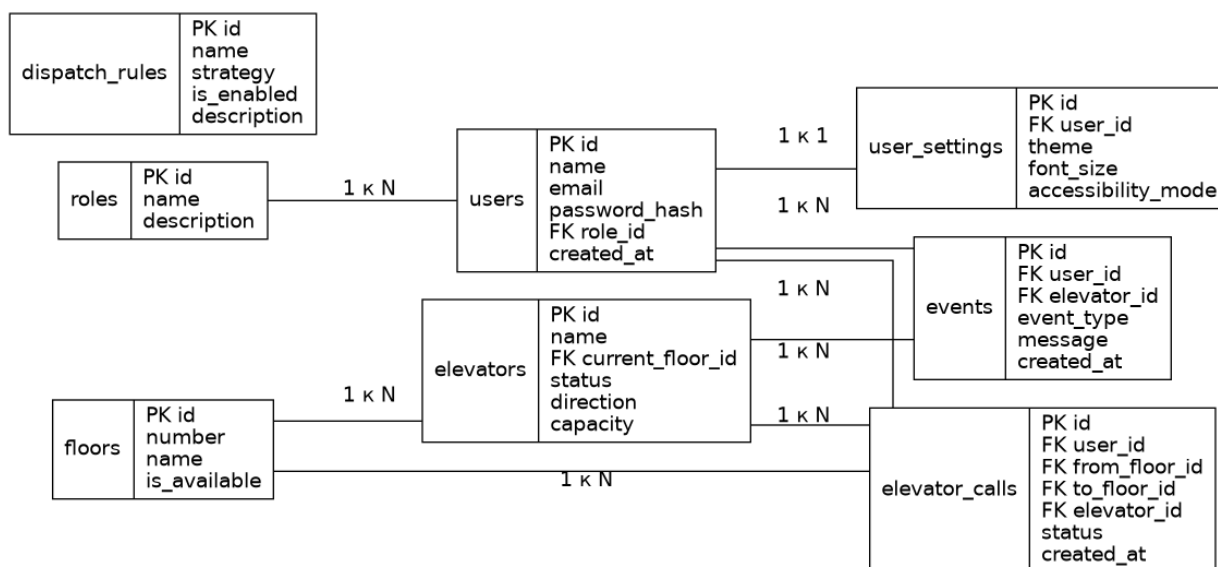


Рисунок 2 - ERD базы данных в нотации Crow's Foot

Таблица	Назначение	Ключевые поля
roles	Справочник ролей пользователей.	id, name.
users	Учетные записи пользователей.	id, email, password_hash, role_id.
user_settings	Персональные	id, user_id, theme,

	настройки интерфейса.	font_size.
floors	Этажи здания.	id, number, name, is_available.
elevators	Лифты и их текущие состояния.	id, name, current_floor_id, status, direction.
elevator_calls	Вызовы лифта.	id, user_id, from_floor_id, to_floor_id, elevator_id, status.
events	Журнал событий.	id, user_id, elevator_id, event_type, message.
dispatch_rules	Правила и стратегии диспетчеризации.	id, name, strategy, is_enabled.

8 UML-проектирование

8.1 Use Case диаграмма

Use Case диаграмма показывает взаимодействие основных акторов с системой: пассажира, диспетчера, администратора и технического специалиста.

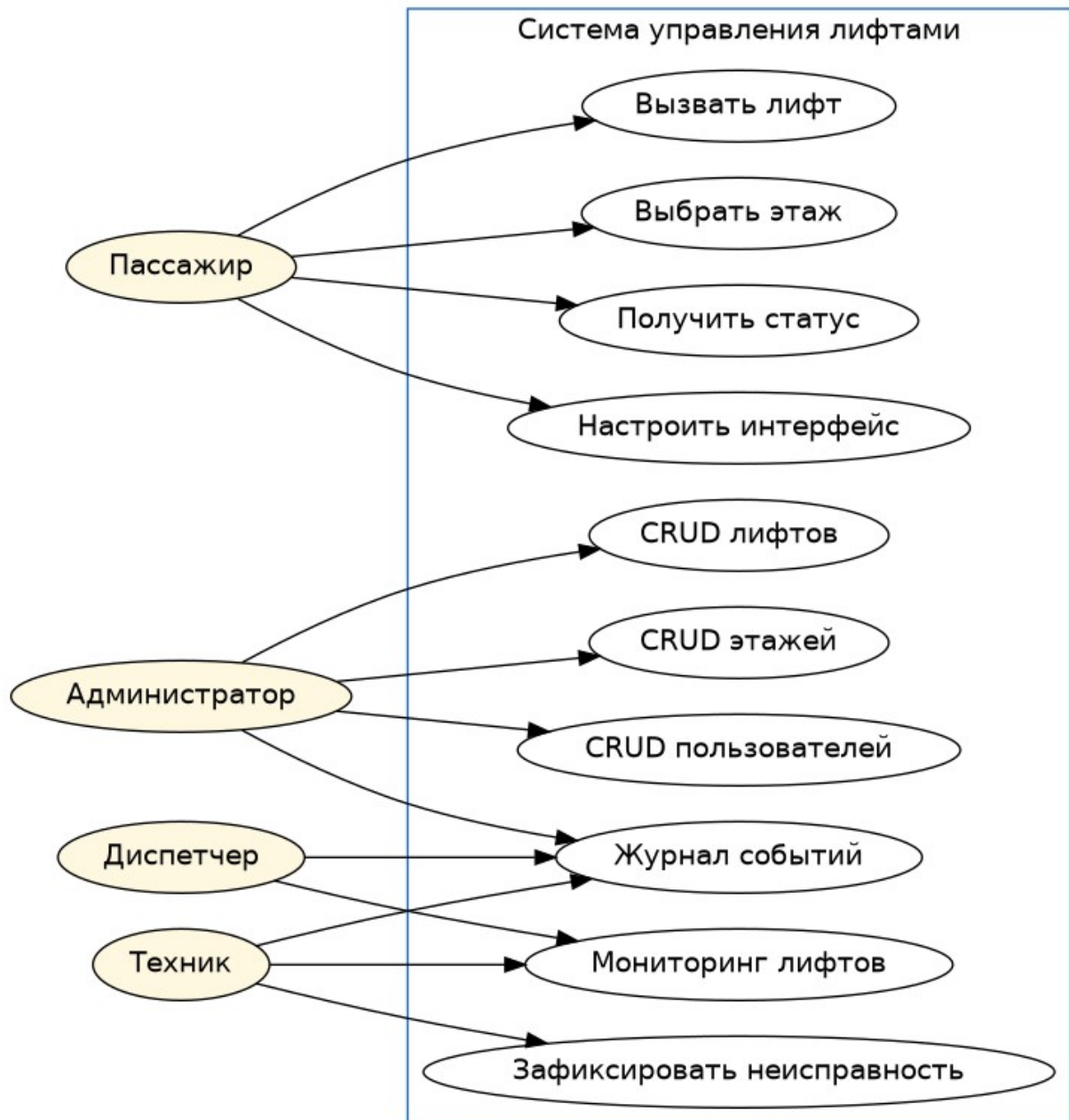


Рисунок 3 - Use Case диаграмма системы

8.2 Диаграмма активности

Диаграмма активности описывает процесс создания вызова лифта, проверку корректности данных, назначение кабины и отображение результата пользователю.



Рисунок 4 - Диаграмма активности процесса вызова лифта

8.3 Диаграмма процессов

Процесс обработки вызова включает последовательные этапы от приема запроса до записи события в журнал и возврата ответа пользователю.

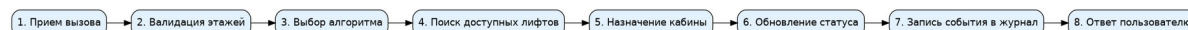


Рисунок 5 - Диаграмма процесса обработки вызова

9 Паттерны проектирования

В проекте применены три паттерна проектирования: Strategy, Observer и Repository. Они позволяют разделить алгоритмы диспетчеризации, организовать уведомления о событиях и изолировать доступ к базе данных от бизнес-логики.

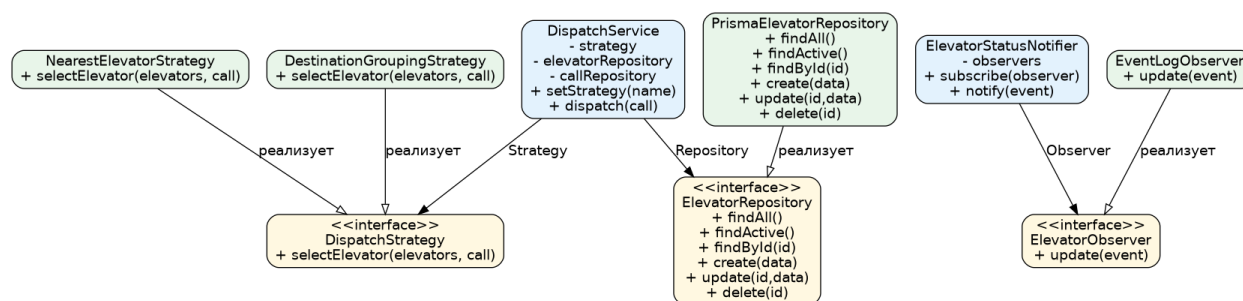


Рисунок 6 - Диаграмма классов с паттернами проектирования

Паттерн	Где используется	Польза
Strategy	DispatchService использует	Можно менять алгоритм выбора лифта без

	NearestElevatorStrategy или DestinationGroupingStrategy.	изменения контроллеров.
Observer	ElevatorStatusNotifier уведомляет EventLogObserver о важных событиях.	Журналирование отделено от основной бизнес-логики.
Repository	PrismaElevatorRepository и PrismaCallRepository скрывают работу ORM.	Сервисы не зависят от конкретных SQL- запросов.

10 Проектирование интерфейса

Интерфейс проектировался с учетом единообразия, простоты освоения и доступности. Основные экраны имеют одинаковую структуру: боковое меню, верхнюю панель, рабочую область и блоки обратной связи. Для статусов применяются цветовые индикаторы и текстовые подписи, чтобы пользователь не зависел только от цвета.

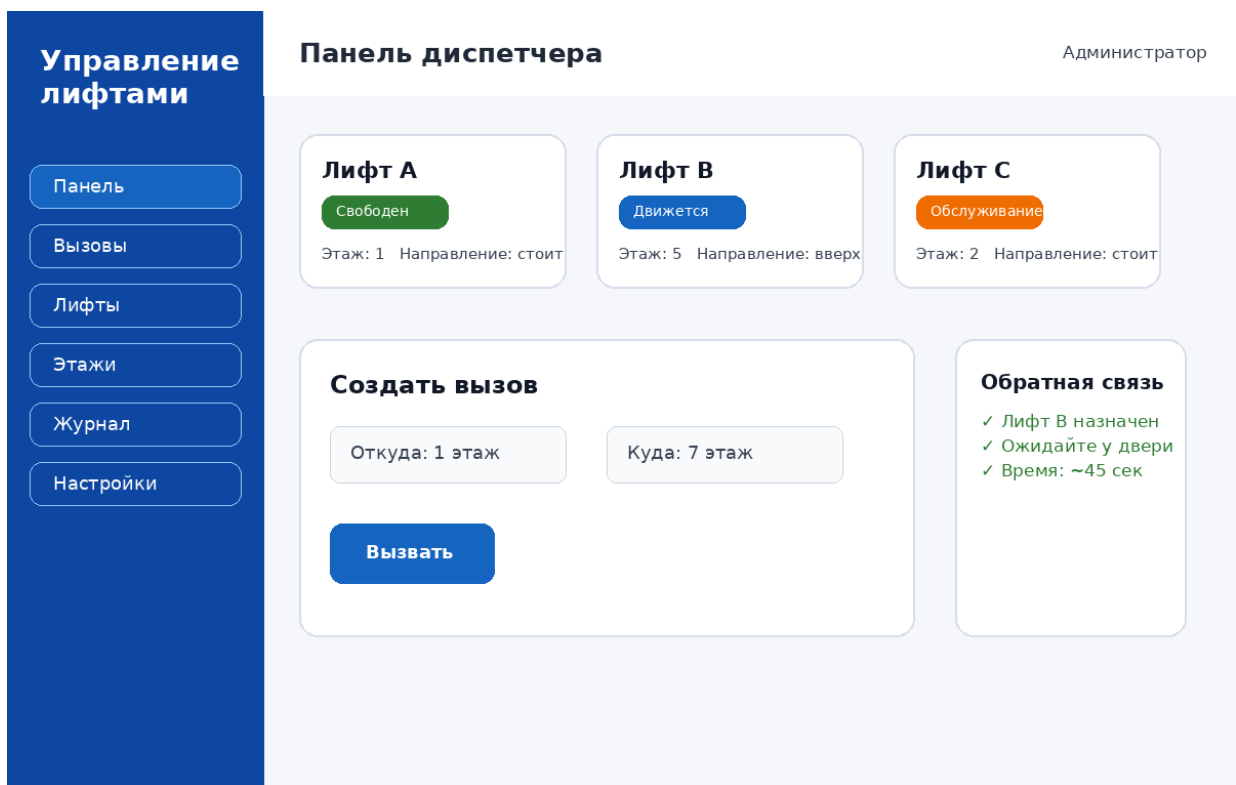


Рисунок 7 - Мокап панели диспетчера

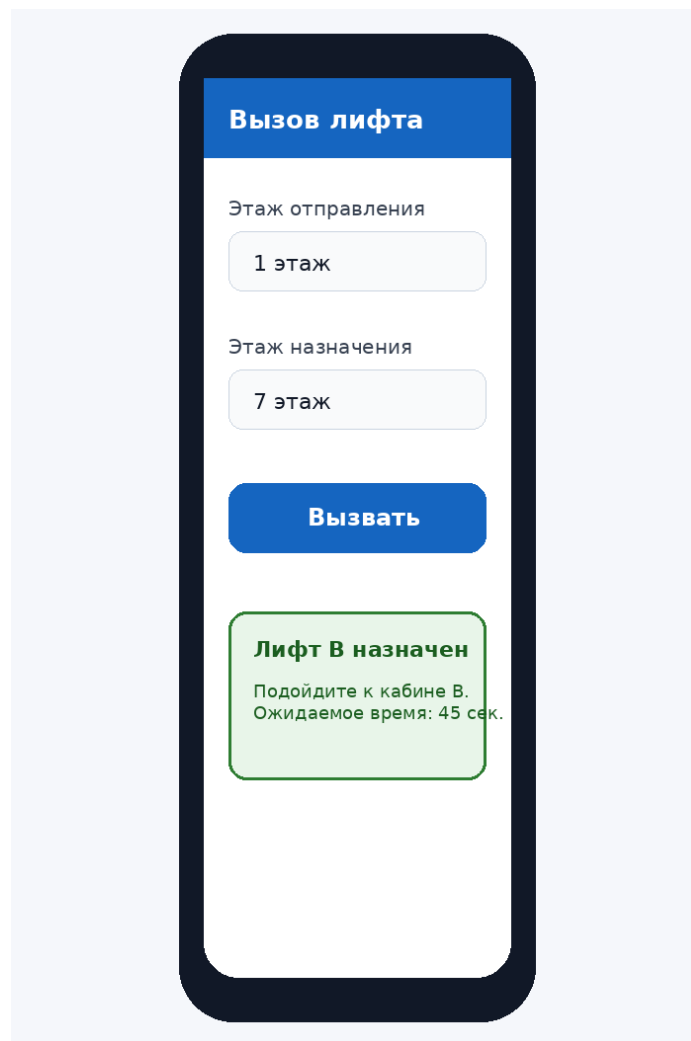


Рисунок 8 - Мокап мобильного сценария вызова лифта

Режим доступности

Крупный текст

Размер текста увеличен для слабовидящих пользователей

Высокий контраст

Основные элементы имеют четкую границу и контраст

Понятная обратная связь

Успешные действия и ошибки отображаются явно

Настройки пользователя

Интерфейс адаптируется под потребности пользователя

Рисунок 9 - Мокап режима доступности

Принцип интерфейса	Реализация
Единообразие	Одинаковое расположение меню, заголовков, таблиц и кнопок.
Обратная связь	Сообщения об ошибке или успехе выводятся сразу после действия.
Настраиваемость	Пользователь выбирает тему, размер шрифта и режим доступности.
Доступность	Контрастный режим, крупные элементы, понятные подписи.
Интуитивность	Основные действия названы естественными глаголами: вызвать, сохранить, удалить.

11 REST API и CRUD

REST API предоставляет frontend доступ к основным данным системы. Все ответы передаются в формате JSON. Для защищенных запросов используется JWT-токен, который frontend передает в заголовке Authorization.

Метод	Endpoint	Назначение
POST	/api/auth/login	Авторизация пользователя.
GET	/api/users	Получение пользователей.
POST	/api/users	Создание пользователя.
PUT	/api/users/:id	Редактирование пользователя.
DELETE	/api/users/:id	Удаление пользователя.
GET	/api/elevators	Получение списка лифтов.
POST	/api/elevators	Создание лифта.
PUT	/api/elevators/:id	Редактирование лифта.
DELETE	/api/elevators/:id	Удаление лифта.
POST	/api/calls	Создание вызова лифта.
PATCH	/api/calls/:id/cancel	Отмена вызова.
GET	/api/events	Просмотр журнала событий.

GET/PUT	/api/settings	Получение и сохранение настроек интерфейса.
---------	---------------	---

CRUD реализован для сущностей users, elevators, floors, dispatch_rules. Для elevator_calls доступны создание, просмотр и отмена, так как полное удаление вызовов нежелательно с точки зрения аудита.

12 Реализация приложения

12.1 Структура проекта

```
elevator-control-course/
├── frontend/      # Vue 3 приложение
│   └── src/
│       ├── api/      # Axios-клиент
│       ├── components/ # Общие UI-компоненты
│       ├── pages/     # Экраны приложения
│       ├── router/    # Маршрутизация
│       └── stores/     # Pinia-хранилища
├── backend/       # REST API
│   ├── prisma/      # Схема базы данных и seed
│   └── src/
│       ├── controllers/ # Контроллеры API
│       ├── services/    # Бизнес-логика
│       ├── repositories/ # Доступ к данным
│       ├── routes/      # Маршруты Express
│       └── middlewares/ # Авторизация и ошибки
└── docker-compose.yml
```

12.2 Пример бизнес-логики диспетчеризации

```
export class DispatchService {
  constructor(strategyName = 'nearest') {
    this.elevatorRepository = new PrismaElevatorRepository();
    this.callRepository = new PrismaCallRepository();
    this.strategy = strategies[strategyName] || strategies.nearest;
  }

  async dispatch(call) {
    const elevators = await this.elevatorRepository.findActive();
    const elevator = this.strategy.selectElevator(elevators, call);
    if (!elevator) {
      return this.callRepository.update(call.id, {
        status: 'ERROR',
        errorText: 'Нет доступных лифтов'
      });
    }
  }
}
```

```

    }
    return this.callRepository.update(call.id, {
      elevatorId: elevator.id,
      status: 'ASSIGNED'
    });
  }
}

```

12.3 Пример frontend-запроса

```

async function createCall() {
  const response = await api.post('/calls', {
    fromFloorId: fromFloorId.value,
    toFloorId: toFloorId.value,
    strategy: strategy.value
  });
  message.value = `Вызов создан. Назначен лифт: ${response.data.elevator?.name}`;
}

```

13 Тестирование

Тестирование проводится для frontend и backend. Проверяются основные бизнес-процессы: авторизация, CRUD-операции, создание вызова, назначение лифта, обработка некорректных данных, отображение статусов и сохранение пользовательских настроек.

№	Тест-кейс	Действия	Ожидаемый результат
ТС-01	Авторизация	Ввести email и пароль администратора.	Пользователь попадает на панель диспетчера.
ТС-02	Создание вызова	Выбрать этаж отправления и назначения, нажать «Вызвать».	Система создает вызов и назначает лифт.
ТС-03	Ошибка выбора	Выбрать	Появляется

	этажа	одинаковые этажи.	понятное сообщение об ошибке.
ТС-04	CRUD лифтов	Создать, изменить и удалить лифт.	Изменения отображаются в таблице.
ТС-05	CRUD этажей	Создать, изменить и удалить этаж.	Данные сохраняются в базе.
ТС-06	Ролевая модель	Зайти как пассажир и открыть пользователей.	Доступ запрещен.
ТС-07	Журнал событий	Создать вызов и изменить лифт.	В журнале появляются события.
ТС-08	Настройки интерфейса	Включить крупный шрифт и режим доступности.	Интерфейс меняет внешний вид.

На backend проверяются коды ответов API, валидация входных данных, работа JWT и корректность связей в базе данных. На frontend проверяются маршруты, формы, сообщения обратной связи и корректное отображение данных из API.

14 Введение в эксплуатацию

Для запуска проекта необходимо установить Docker и выполнить команду `docker compose up --build` в корневой папке проекта. Docker Compose поднимает три контейнера: PostgreSQL, backend API и frontend-приложение. При первом запуске выполняется миграция базы данных и заполнение демонстрационными данными.

```
docker compose up --build
```

После запуска frontend доступен по адресу `http://localhost:5173`, backend API - по адресу `http://localhost:3000/api`. Для входа можно использовать демонстрационную учетную запись администратора `admin@example.com` с паролем `admin123`.

14.1 Требования к среде

Компонент	Минимальная версия	Назначение
Docker	24+	Контейнеризация приложения.
Docker Compose	2+	Оркестрация контейнеров.
Node.js	20+	Локальная разработка frontend/backend.
PostgreSQL	16+	Хранение данных.

Заключение

В ходе курсовой работы разработана информационная система управления лифтами. Была проанализирована предметная область, определены роли пользователей и ключевые объекты данных, сформулированы

функциональные и нефункциональные требования. В соответствии с методологией Waterfall выполнены этапы анализа, проектирования, реализации, тестирования и подготовки к эксплуатации.

Практическая реализация выполнена в виде распределенного монолита: frontend на Vue 3 отделен от backend REST API на Node.js/Express, обмен данными осуществляется в формате JSON. В системе реализованы CRUD-операции, авторизация JWT, разграничение доступа по ролям, алгоритм выбора лифта, журнал событий, настройки интерфейса и режим доступности.

Для проектирования подготовлены UML-диаграммы, диаграмма процессов, диаграмма активности, Use Case-диаграмма и ERD базы данных. В архитектуре применены паттерны Strategy, Observer и Repository, что повышает расширяемость и сопровождаемость системы. Поставленная цель достигнута, а разработанный проект может быть использован как учебный прототип системы диспетчеризации лифтов.

Список источников

1. Методические требования дисциплины «Создание программного обеспечения» к курсовым работам и проектам.
2. ГОСТ Р ИСО 9241-11-2010. Эргономика взаимодействия человек-система.
3. ГОСТ Р ИСО 9241-110-2008. Принципы организации диалога.
4. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы.
5. Документация Vue.js: основные принципы разработки клиентских приложений.
6. Документация Express.js: разработка REST API.
7. Документация Prisma ORM: описание моделей и миграций базы данных.

Приложение А. Фрагмент схемы Prisma

```
model Elevator {  
  id      Int      @id @default(autoincrement())  
  name    String   @unique  
  currentFloorId Int  
  currentFloor Floor @relation("CurrentFloor", fields: [currentFloorId], references: [id])  
  status  ElevatorStatus @default(IDLE)  
  direction Direction @default(STOPPED)  
  capacity Int      @default(8)  
  maxFloor Int      @default(10)  
  isActive Boolean   @default(true)  
  calls   ElevatorCall[]  
  events  Event[]  
}
```

Приложение Б. Инструкция пользователя

Для работы пользователь должен войти в систему. После авторизации он попадает на панель диспетчера. На панели отображаются карточки лифтов и форма создания вызова. Для создания вызова необходимо выбрать этаж отправления, этаж назначения и нажать кнопку «Вызвать». Если операция успешна, система покажет назначенный лифт. Если данные некорректны, пользователь увидит понятное сообщение об ошибке.

Администратор может перейти в разделы «Лифты», «Этажи» и «Пользователи» для выполнения CRUD-операций. В разделе «Журнал» отображаются события системы. В разделе «Настройки» пользователь может включить крупный текст, режим доступности и изменить тему интерфейса.